Kevin: platform, frontend and demo work doc

Oct 24, 2026 · @Chaudhari kevin

Mission and setup

You make Rally live and public: state in Convex, the app on a public URL, real testnet deposits through x402, and every teammate's backend feature visible on screen. You also own the contracts, the merges and the demo.

- Branch: `feat/platform-live` (plus `chore/contracts`, merged first).
- You own: `convex/`, `apps/web/`, `services/api/rally/store.py`, `services/api/rally/main.py`, `packages/contracts/`, a new `infra/`, `README.md`, `.env.example`.
- Don't edit: `negotiation/`, `agents/`, `compiler.py`, `matching.py`, `ai/`.

Setup:

```
git checkout main && git pull
python scripts/rally.py setup
npm install
npx convex dev            # log in, create the deployment, writes CONVEX_URL
npx convex env set RALLY_SERVER_KEY <long random string>
# .env: STORE=convex  CONVEX_URL=...  RALLY_SERVER_KEY=<same>
# apps/web/.env.local: NEXT_PUBLIC_CONVEX_URL=...
NEXT_PUBLIC_API_URL=http://localhost:8000
pip install convex
python scripts/rally.py dev
```

What exists already: `store.py` has `ConvexMirror` (writes rows and one snapshot per deal through server-key-guarded mutations in `convex/rally.ts`), and `apps/web/lib/useDeal.ts` switches to a Convex subscription when `NEXT_PUBLIC_CONVEX_URL` is set. Neither has run against a real deployment yet. The vendor portal still polls the API.

Tasks, in this order

#	Task	Priority	Hours	Due
0	Confirm sponsor rules and the deadline at the booths	P0	0.5	3 pm
1	Contracts PR into `main`	P0	1	3:30 pm
2	Convex live state	P0	4	9 pm, merge 1
3	Public deployment: Vultr + Vercel	P0	3	9 pm
4	x402 testnet deposits	P1	4	2 am, merge 2
5	UI for Parikshit's and Dev's features	P1	3	2 am
6	Negotiation replay scrubber	P1	3	2 am
7	Calendar invite and share card	P2	1	only if time
8	README, video, Devpost	P0	4	9 am Sunday

1. Contracts PR

On a `chore/contracts` branch, add `Deal.nudge` (`providerId`, `extraPeople`, `newPricePerPerson`, `savingPerPerson`, or null) and `Deal.replacements` (list of `leftUserId`, `invitedUserId`, `at`) to `packages/contracts/rally_contracts/deal.py`; add the new env flags to `.env.example`. Run `python scripts/rally.py contracts` and `test`, merge, and tell the team to branch.

2. Convex live state

1. Run the API with `STORE=convex` and check that `/v1/health` shows Convex enabled with 0 errors.
2. Set `NEXT_PUBLIC_CONVEX_URL` and confirm the group room updates with no polling. Open it on two laptops at once.
3. Mirror the vendor view: write one `vendor:<providerId>:<dealId>` document per vendor through `putRow`, and switch `/vendor/requests/[dealId]` to a Convex subscription (keep polling as the fallback).

4. Restart safety: on API startup with `STORE=convex`, reload deals, members, proposals, events, agent messages and chat from Convex rows, so killing the API mid-demo loses nothing. This turns the mirror into the real system of record.
5. Membership check on reads: pass the persona token to `dealSnapshot` and verify it in the query, so a snapshot can't be read by guessing a deal id.

Test: an integration test (skipped without `CONVEX_URL`) writes an event from Python and reads it back from the query. Manual: kill and restart the API mid-negotiation; the room still shows every round.

3. Public deployment

1. Vultr VM (Ubuntu): `infra/vultr/` with systemd units for `uvicorn rally.main:app` and Parikshit's `agents/run_agents.py`, and Caddy in front for HTTPS on a domain or the VM's hostname.
2. Vercel for `apps/web` with `NEXT_PUBLIC_API_URL` and `NEXT_PUBLIC_CONVEX_URL`. Add the Vercel URL to the API's `CORS_ORIGINS`.
3. Set `RALLY_WEB_URL` for Parikshit's concierge links.
4. Never commit keys: set them in the VM's env file, Vercel and Convex.

Done when a judge's phone opens the Vercel URL and finishes the scuba flow.

4. x402 testnet deposits

1. Read the current x402 docs (x402.org and Coinbase's CDP x402 pages) for the Python server middleware and client. The earlier plan noted: the server answers 402 with payment requirements, the client signs and retries, and a facilitator settles on chain; Base Sepolia is the test network; payments are not escrow and refunds are separate transfers.
2. Put `POST /v1/deals/{id}/commitments` behind x402 when `PAYMENTS=x402_testnet`, priced at a small deposit (for example 1 test USDC). Refuse any mainnet network id at startup (the code already refuses unknown `PAYMENTS` values).
3. Payer: one testnet wallet per demo persona held by the server and funded from a faucet. The human's Commit click is the authorization; agents never pay on their own.
4. Store the transaction hash in `Commitment.txRef`, link it to the Base Sepolia explorer on the confirmation page, and label it "Testnet deposit".

Fallback: `PAYMENTS=simulated`, already working.

5. UI for teammates' features

Feature	Owner of the backend	Where it shows
Real agent addresses, "Agents: agentverse" badge	Parikshit	Group room members and agent conversation, Agents page
Group-size nudge card with "Invite one more"	Parikshit	Group room, above the plan
Replacement note ("Ana left; Eli joined")	Parikshit	Group room members, confirmation summary
"Why this plan works for you" card	Dev	Group room and approve page, private to the viewer
"Compiled by Gemini" and "Ranked by Moss" labels	Dev	Review and matches pages (already wired to the API fields)
Vendor search box	Dev	Home page
"Listed price, not a quote" vendors	Dev	Provider comparison page
"Also on ASI:One" link with the concierge's address	Parikshit	Home page

6. Negotiation replay

A scrubber under the round timeline that steps through plan versions v1 to vN, showing that version's terms, each agent's response and the reason for the next change. It's built only from snapshot data (`proposals`, `rounds`, `agentMessages`), so no backend work is needed.

7. Calendar invite and share card (P2)

On the confirmation page: a downloadable `.ics` for the agreed date and a copyable plan summary.

8. README, video and Devpost

README: architecture, how each sponsor is used, agent names and addresses, both Innovation Lab badges, measured results, how to run. Record the 3 to 5 minute video from the demo script in the team plan. Submit at least 2 hours before the deadline.

Merging, booth questions and checklist

Merge order at each gate (9 pm and 2 am): `feat/platform-live` first, then `feat/agents-network`, then `feat/ai-search-data`, each rebased on the previous merge. After every merge run `python scripts/rally.py test` and `node tests/e2e/two_sided_walk.js` (Playwright) before announcing "main is green".

Questions for the sponsor booths (before 3 pm; write the answers into `docs/prize-rules.md`):

Sponsor	Ask
Fetch.ai	Same rules as their LA Hacks 2026 track (Agentverse registration, Chat Protocol, ASI:One session URL, README badges, 3–5 min video)? Is a custom frontend allowed alongside ASI:One?
Google (Gemini)	Is there a Gemini track, and does it need a specific model or API?
Moss	Prize criteria, and can we get project keys today?
Convex	Must Convex be the system of record, or is a live mirror enough?
Apify	Must the Actor be our own, or can we use a store Actor?
Coinbase (x402)	Testnet acceptable? Which facilitator and network?
Vultr	Credits, and does hosting the API on a VM qualify?
Visa	Is a Visa API expected? If so, which one?
Organizers	Exact submission deadline and the demo/judging format

Fallbacks:

- Convex fails: keep `STORE=memory`; the room already polls the API.
- Vultr or Vercel fails: run locally and use the venue network, or a tunnel to the laptop.
- x402 fails: `PAYMENTS=simulated` with its "Simulated, no money moved" label.

Final checklist:

- [ ] Contracts PR merged by 3:30 pm and the team branched
- [ ] `STORE=convex`: group room and vendor portal update live on two laptops
- [ ] API restart mid-negotiation loses nothing
- [ ] Public URLs work from a phone
- [ ] x402 testnet deposit with a transaction link, or the simulated fallback clearly labelled
- [ ] Every teammate feature visible in the UI (table in task 5)
- [ ] Replay scrubber works for all three activities
- [ ] README complete; Devpost video uploaded; submitted 2 hours early

The team plan has the feature list, timeline and demo script; Parikshit's and Dev's work docs list what they will hand you and when.